Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number: 2520030207
Name: V. PRANAV

AOOP
25CS2205E

2026

Case Study 4: Hotel Reservations and Date Ranges

Case Study:

A hotel chain needs a Java application for room search, reservation, confirmation, check-in, check-out and cancellation. Existing failures include check-out dates before check-in dates, overlapping reservations for the same room, confirmed reservations without payment guarantees, full refunds after check-in and cancellation after the permitted stage. The design must model valid date ranges, room availability, payment guarantees and reservation state transitions so invalid combinations cannot be created or persisted.

Q1. Domain Invariants and Value Objects (6 Marks)

The following invariants should always hold for a valid reservation.

1. Valid stay period: Check-in must be strictly earlier than check-out, so every stay includes at least one night. **Importance / consequence:** Rejects zero-night and reversed stays.

2. No overlapping booking for one room: Two active reservations assigned to the same room must not overlap in date range. **Importance / consequence:** Prevents double booking and customer disputes.

3. Half-open date interpretation: Treat a stay as [check-in, check-out): check-in is included and check-out is excluded. **Importance / consequence:** Allows one guest to check out on the same date the next guest checks in.

4. Room assignment: A confirmed reservation must refer to a valid room and guest. **Importance / consequence:** Prevents incomplete bookings from being treated as confirmed.

5. Payment guarantee: A reservation cannot be confirmed until the required payment guarantee is recorded. **Importance / consequence:** Prevents unsupported confirmed bookings.

6. Lifecycle correctness: Only permitted transitions are allowed: requested, confirmed, checked in, completed or cancelled. **Importance / consequence:** Prevents cancellation after check-in and contradictory states.

7. Check-in/check-out rules: Check-in is permitted only for a confirmed reservation at the permitted time; completion follows check-in and check-out. **Importance / consequence:** Prevents staff from skipping lifecycle stages.

8. Cancellation and refund policy: Cancellation must be allowed by the current state and refund must be calculated using the applicable policy. **Importance / consequence:** Prevents full refunds when the policy does not allow them, especially after check-in.

Value objects for this domain

StayPeriod / DateRange: Immutable check-in and check-out dates. Equality and overlap depend on the two dates.

RoomIdentifier and ReservationIdentifier: Validated immutable identifiers used as values; they do not own a lifecycle.

Money: Amount plus currency for price, deposit, charge and refund calculations.

PaymentGuarantee: An immutable reference to the payment authorization, provider and guaranteed amount.

CancellationPolicy: The policy terms or version used to calculate cancellation charges and refunds.

Q2. Designing DateRange and Reservation (8 Marks)

Represent a stay with java.time.LocalDate. The range is half-open: the check-in date is included, while the check-out date is not occupied by the leaving guest. A valid stay must contain at least one night.

Java StayPeriod value object

```
import java.time.LocalDate;
import java.util.Objects;

public record StayPeriod(LocalDate checkIn, LocalDate checkOut) {
    public StayPeriod {
        Objects.requireNonNull(checkIn, "check-in required");
        Objects.requireNonNull(checkOut, "check-out required");
        if (!checkOut.isAfter(checkIn))
            throw new IllegalArgumentException("Check-out must be after check-in");
    }

    // Half-open intervals: [checkIn, checkOut)
    public boolean overlaps(StayPeriod other) {
        Objects.requireNonNull(other);
        return checkIn.isBefore(other.checkOut())
            && other.checkIn().isBefore(checkOut);
    }

    public long nights() {
        return java.time.temporal.ChronoUnit.DAYS.between(checkIn, checkOut);
    }
}
```

How overlap comparison works

Two ranges overlap only when the first begins before the second ends AND the second begins before the first ends. In formula form: startA < endB AND startB < endA.

Example 1: 10 June to 12 June and 12 June to 15 June do not overlap. The first guest checks out on 12 June and the next guest checks in that day. Example 2: 10 June to 13 June and 12 June to 15 June overlap because both occupy the night/date interval beginning 12 June.

The constructor prevents invalid dates from being represented at all. Because the fields are final record components and there are no setters, the range cannot later be changed into an invalid range. The overlap method applies one consistent date convention everywhere.

Q3. Entity Classification and Aggregate Design (6 Marks)

Guest - Entity: A guest has a stable guest ID and may update contact details over time.

Room - Entity: A room has a stable room identifier and a lifecycle/operational status such as available, occupied or out of service.

Reservation - Entity: A reservation has a reservation ID and changes through requested, confirmed, checked-in, completed or cancelled states.

Room Identifier - Value Object: An immutable identifier value; equality is based on its normalized value.

Reservation Identifier - Value Object: An immutable identifier used to refer to a reservation.

Stay Period - Value Object: An immutable date range whose equality is based on check-in and check-out dates.

Cancellation Policy - Value Object: A versioned set of terms used as a value in refund calculation; it does not need independent identity for the reservation decision.

UML-style domain relationship (text representation)

```
Guest 1 ---------------- 0..* Reservation
Room  1 ---------------- 0..* Reservation
Reservation
  - reservationId: ReservationIdentifier
  - guest: Guest
  - room: RoomIdentifier
  - stay: StayPeriod
  - state: ReservationState
  - guarantee: PaymentGuarantee
  - policy: CancellationPolicy

RoomSchedule / RoomInventory (availability consistency boundary)
  - roomId: RoomIdentifier
  - activeReservations: reservations for this room
  + reserve(stay): checks overlap and records allocation

Reservation (lifecycle boundary)
  + confirm(guarantee)
  + checkIn(date)
  + checkOut(date)
  + cancel(at)
```

Recommended consistency boundary: the room schedule or room-inventory aggregate controls allocation for a particular room and checks all active reservations for overlap. A reservation aggregate controls the lifecycle and its own payment/guest/stay data. Room allocation must be performed through the room schedule, not by independently creating reservation records. In the database, the allocation check and insert must occur in one protected transaction.

Q4. Reservation Lifecycle and Making Illegal States Unrepresentable (6 Marks)

Permitted transitions

- REQUESTED -> CONFIRMED, after all required details and a payment guarantee are available.
- REQUESTED -> CANCELLED, when the request is withdrawn before confirmation.
- CONFIRMED -> CHECKED_IN, when the guest is eligible to check in.
- CONFIRMED -> CANCELLED, only when the cancellation policy and timing allow it.
- CHECKED_IN -> COMPLETED, after the guest checks out.

Prohibited transitions

- REQUESTED -> CHECKED_IN without confirmation.
- REQUESTED or CONFIRMED -> COMPLETED without check-in and check-out.
- CHECKED_IN -> CANCELLED through the ordinary cancellation operation.
- COMPLETED -> CONFIRMED or CHECKED_IN.
- CANCELLED -> any active state. A new stay requires a new reservation.

Information required at confirmation: Reservation ID, guest, room, valid StayPeriod, agreed price/currency, payment guarantee reference and amount, cancellation-policy version and any required guest/contact details. Confirmation should not succeed if a required item is missing.

Preventing cancellation after check-in: The cancel method checks the current state and accepts only REQUESTED or eligible CONFIRMED reservations. A CHECKED_IN reservation must use the check-out/completion process; refunds or service recovery after that point must be handled as a separately authorized adjustment, not by changing it to CANCELLED.

Preventing contradictory states: Do not expose a public status setter. Use a private state field and transition methods that validate the current state. Store required data together with the state that needs it; for example, confirmation requires a non-null PaymentGuarantee. For stronger compile-time safety, use state-specific classes such as RequestedReservation, ConfirmedReservation and CheckedInReservation, each exposing only valid operations.

Illustrative lifecycle guard

```
enum ReservationState { REQUESTED, CONFIRMED, CHECKED_IN, COMPLETED, CANCELLED }

void confirm(PaymentGuarantee guarantee) {
    if (state != ReservationState.REQUESTED)
        throw new IllegalStateException("Only requested bookings can be confirmed");
    this.guarantee = Objects.requireNonNull(guarantee);
    state = ReservationState.CONFIRMED;
}

void cancel() {
    if (state != ReservationState.REQUESTED && state != ReservationState.CONFIRMED)
        throw new IllegalStateException("Cancellation not permitted in this state");
```

```
    // Apply the relevant cancellation policy before finalizing cancellation.
    state = ReservationState.CANCELLED;
}
```

Q5. Concurrent Reservations and Cancellation Policies (4 Marks)

1. Concurrent booking problem: Both receptionists may read the room as available before either saves a reservation. Each then creates a booking, causing two overlapping reservations for one room. This is a race condition / double-booking problem.

2. Protection: Perform availability check and reservation allocation within one database transaction. Lock the room or its schedule row, or use serializable isolation so competing requests are ordered. Recheck overlap after acquiring the lock. The losing request must receive an unavailable-room response.

3. Refund calculation: Use the cancellation policy agreed for that reservation, including the cancellation timestamp, time before check-in, amount already paid, non-refundable fees and any applicable penalty. A simple model is refund = amount paid - cancellation fee, bounded so it cannot be negative. After check-in, the standard policy should not issue a full cancellation refund; any exception needs an explicit adjustment/approval rule.

4. Persistence safeguards: Use foreign keys for guest/room/reservation references, transactions and row locks or serializable isolation, a version column for optimistic locking, uniqueness for reservation IDs, and a database exclusion/overlap constraint where the database supports range constraints. An ordinary unique index on room ID alone is not sufficient because it cannot by itself detect arbitrary overlapping date ranges.

Common team submission and presentation checklist

- Problem analysis: explain invalid dates, double bookings, missing guarantees and cancellation failures.
- Invariant identification: present date, availability, confirmation, payment, lifecycle and refund rules.
- Domain classification: justify Guest, Room, Reservation, identifiers, StayPeriod and CancellationPolicy.
- UML class diagram: show Guest, Room, Reservation, StayPeriod, PaymentGuarantee and RoomSchedule.
- Java implementation: demonstrate valid/invalid periods and the overlap test.
- Design justification: explain half-open ranges, aggregate boundaries, state transitions and persistence-level concurrency controls.
- Team presentation: demonstrate two simultaneous requests for one room and show that only one succeeds.